BharatQuest MVP
Technical Design Document

Expo + React Native | 3-Day Hackathon Build

Derived from the provided PRD for the Golden Path demo.

1. Purpose and build philosophy

This document defines the technical shape of the BharatQuest MVP as a single-path, demo-safe Expo application. The goal is not product completeness; it is a reliable live demo with predictable state transitions, local persistence, scripted dialogue, and no external runtime dependencies beyond Expo-managed packages.

The PRD explicitly frames the app as a six-state Golden Path with a hardcoded starting balance of ₹5,000, a Trust Score of 80/100, a fake UPI reward scam, a forced decision point, consequence feedback, and a final sync fake-out. That structure is the backbone of this design.

2. Scope locked for the hackathon

- React Native with Expo; stay inside the managed workflow.
- React Context for session/game state.
- AsyncStorage only for persisting Balance and Trust Score locally.
- expo-speech for text-to-speech narration of NPC messages.
- lottie-react-native for sync and result animations.
- No backend, no live APIs, no LLM calls, no SQLite/WatermelonDB, no ejecting.
- All dialogue, UI labels, and scenario content live in local JSON/TS objects.

3. System architecture

The app is intentionally thin. The data flow is: local scripted content -> context state -> screen components -> AsyncStorage persistence -> optional speech playback. The only side effects are local storage writes, timed UI transitions, and narration.

Recommended top-level shape:

```
App.tsx
src/
  assets/
    lottie/
    images/
    audio/                # optional fallback UI sounds if used
  components/
    common/
      Badge.tsx
      HUDStat.tsx
      PrimaryButton.tsx
      ScreenShell.tsx
      VoiceFab.tsx
    game/
```

```
      FarmBackdrop.tsx
      RewardBanner.tsx
      ScamChat.tsx
      DecisionSheet.tsx
      ConsequenceModal.tsx
      SyncOverlay.tsx
  config/
    content.ts            # hardcoded story beats and UI copy
    theme.ts              # colors, spacing, typography
  context/
    GameContext.tsx
  hooks/
    useVoiceNarration.ts
    useGameTimers.ts
  navigation/
    RootNavigator.tsx
  screens/
    DashboardScreen.tsx
    RewardPopupScreen.tsx
    ChatScreen.tsx
    DecisionScreen.tsx
    ResultScreen.tsx
  services/
    storage.ts
    mockSync.ts
  types/
    game.ts
```

This structure keeps business logic separate from presentation without creating too many files for a 3-day build. The app should be boring in the best possible way: easy to find, easy to debug, hard to break.

4. Data and state model

The app should keep all runtime game state in React Context, then persist only the stable user metrics and completion flags in AsyncStorage.

State field	Type	Purpose
balance	number	Current rupee balance shown in the HUD.
trustScore	number	Current trust score shown in the HUD.
flowStep	0-5 enum	Controls which of the six screens is active.
chatIndex	number	Tracks the current scripted NPC message.
hasSeenRewardBanner	boolean	Prevents the reward modal from replaying unexpectedly.
decision	'claim' | 'report' | null	Stores the selected branch.
syncStatus	'idle' | 'syncing' | 'success' | 'error'	Controls Lottie overlay state.
voiceEnabled	boolean	Lets the narration hook know whether to speak.

State field	Type	Purpose
lastSpokenMessageId	string | null	Prevents duplicate speech playback.

Recommended React Context shape:

```
export type FlowStep =
  | "dashboard"
  | "reward"
  | "chat"
  | "decision"
  | "result"
  | "sync";

export type Decision = "claim" | "report" | null;

export interface GameState {
  balance: number;
  trustScore: number;
  flowStep: FlowStep;
  chatIndex: number;
  decision: Decision;
  syncStatus: "idle" | "syncing" | "success" | "error";
  voiceEnabled: boolean;
  lastSpokenMessageId: string | null;
  hasSeenRewardBanner: boolean;
}
```

5. AsyncStorage contract

Persist only the minimum durable state: balance, trust score, whether the demo has completed, and the last selected branch. Everything else can safely reset when the app reloads.

Key	Stored value	Notes
bharatquest.balance	number as string or JSON	Persist current currency balance.
bharatquest.trustScore	number as string or JSON	Persist current trust score.
bharatquest.flowStep	string	Optional: resume at the current scripted state.
bharatquest.decision	claim | report	Store the last branch for analytics/demo continuity.
bharatquest.syncComplete	true | false	Used to show the final sync state after the fake cloud step.

Exact AsyncStorage implementation pattern:

```
import AsyncStorage from "@react-native-async-storage/async-storage";

const STORAGE_KEYS = {
  balance: "bharatquest.balance",
  trustScore: "bharatquest.trustScore",
  flowStep: "bharatquest.flowStep",
```

```
  decision: "bharatquest.decision",
  syncComplete: "bharatquest.syncComplete",
} as const;

export async function loadGameSnapshot() {
  const [balanceRaw, trustRaw, flowRaw, decisionRaw, syncRaw] =
    await AsyncStorage.multiGet([
      STORAGE_KEYS.balance,
      STORAGE_KEYS.trustScore,
      STORAGE_KEYS.flowStep,
      STORAGE_KEYS.decision,
      STORAGE_KEYS.syncComplete,
    ]);

  const getValue = (pair: [string, string | null]) => pair[1];

  return {
    balance: Number(getValue(balanceRaw as [string, string | null]) ?? 5000),
    trustScore: Number(getValue(trustRaw as [string, string | null]) ?? 80),
    flowStep: (getValue(flowRaw as [string, string | null]) ?? "dashboard") as FlowStep,
    decision: (getValue(decisionRaw as [string, string | null]) ?? null) as Decision,
    syncComplete: getValue(syncRaw as [string, string | null]) === "true",
  };
}

export async function saveGameSnapshot(snapshot: {
  balance: number;
  trustScore: number;
  flowStep: FlowStep;
  decision: Decision;
  syncComplete: boolean;
}) {
  await AsyncStorage.multiSet([
    [STORAGE_KEYS.balance, String(snapshot.balance)],
    [STORAGE_KEYS.trustScore, String(snapshot.trustScore)],
    [STORAGE_KEYS.flowStep, snapshot.flowStep],
    [STORAGE_KEYS.decision, snapshot.decision ?? ""],
    [STORAGE_KEYS.syncComplete, String(snapshot.syncComplete)],
  ]);
}

export async function resetGameSnapshot() {
  await AsyncStorage.multiRemove(Object.values(STORAGE_KEYS));
}
```

Implementation note: for a hackathon build, multiGet and multiSet are enough. There is no need for a bigger abstraction unless the codebase starts breeding chaos.

6. Component architecture mapped to the 6-step flow

Each state is a screen or overlay, but the experience should feel like one continuous flow. The navigator only switches the active scene; it should not reset the underlying context.

Flow step	Primary component	Key children	Responsibility
1. Dashboard	DashboardScreen	HUDStat, FarmBackdrop, VoiceFab	Show ₹5,000 and 80/100, animate count-up, auto-

Flow step	Primary component	Key children	Responsibility
			advance.
2. Reward trigger	RewardPopupScreen	RewardBanner, dimmed backdrop	Show the fake UPI reward modal, then auto-advance.
3. Scam chat	ChatScreen	ScamChat, VoiceFab	Render scripted NPC bubbles and trigger narration.
4. Decision	DecisionScreen	DecisionSheet, PrimaryButton	Offer Claim Reward vs Report & Block.
5. Result	ResultScreen	ConsequenceModal, HUDStat, VoiceFab	Apply branch outcome and show success/failure education.
6. Sync fake-out	SyncOverlay	LottieView, progress text	Simulate cloud sync with local timeout and animation.

7. Screen-by-screen implementation notes

1. DashboardScreen: render the farm illustration, the fixed HUD, and a floating voice icon. Trigger a count-up animation from initial to final values over 1.5 seconds, then schedule the reward screen after a 3-second delay.
2. RewardPopupScreen: present a top-half modal with hardcoded scam copy and a visible countdown timer. Keep it auto-dismissing after 2.5 seconds.
3. ChatScreen: use a scripted array of messages. Messages appear one by one, with a typing indicator between them. When the phishing link message appears, speak the warning line aloud.
4. DecisionScreen: present two large buttons only. No text input, no extra navigation, no escape hatch.
5. ResultScreen: branch on the selected decision. Claim Reward applies the red flash, balance/trust deduction, and RBI-style educational modal. Report & Block applies the green success feedback and trust gain.
6. SyncOverlay: display a full-screen dimmer plus Lottie animation. The overlay should be controlled entirely by local state and a setTimeout mock request.

8. Suggested component contracts

Shared UI atoms:

- ScreenShell: handles safe area, background color, and common layout spacing.
- HUDStat: reusable metric chip for balance and trust score.
- PrimaryButton: large touch target with icon, label, and branch styling.
- VoiceFab: floating button that can also indicate speaking with a pulse.
- RewardBanner: modal-style scam notice for State 2.
- ScamChat: scripted chat list with avatar, bubble, and typing indicator.
- DecisionSheet: bottom sheet with the two action choices.
- ConsequenceModal: wraps success/failure educational copy.
- SyncOverlay: owns the mocked cloud sync Lottie animation.

9. Mock sync architecture

The sync server is fake by design. The button should call a local function that sets syncStatus to syncing, waits about 2-3 seconds with setTimeout, then resolves to success. The only job of this layer is to give the demo a convincing network moment.

```
export async function mockCloudSync() {
  return new Promise<"success">((resolve) => {
    setTimeout(() => resolve("success"), 2400);
  });
}
```

10. Expo Speech narration hook

Narration should fire once per NPC message, not on every re-render. Keep the hook small, deterministic, and easy to disable for debugging.

```
import { useEffect, useRef } from "react";
import * as Speech from "expo-speech";

type NarrationInput = {
  messageId: string | null;
  text: string;
  enabled: boolean;
  language?: string;
};

export function useVoiceNarration({
  messageId,
  text,
  enabled,
  language = "en-IN",
}: NarrationInput) {
  const lastSpokenRef = useRef<string | null>(null);

  useEffect(() => {
    if (!enabled || !messageId || !text) return;
    if (lastSpokenRef.current === messageId) return;

    Speech.stop();
    Speech.speak(text, {
      language,
      rate: 0.95,
      pitch: 1.0,
      onStart: () => {
        lastSpokenRef.current = messageId;
      },
    });

    return () => {
      // Keep cleanup conservative so rapid message changes do not overlap.
      Speech.stop();
    };
  }, [messageId, text, enabled, language]);
}
```

Suggested usage inside ChatScreen: call useVoiceNarration with the currently visible NPC message. That keeps the voice logic out of the UI tree where it would otherwise turn into spaghetti with better typography.

11. State transitions and timing

Transition	Trigger	Delay	Implementation
Dashboard -> Reward	initial render complete	3.0s	setTimeout in a useEffect or timer hook
Reward -> Chat	reward modal shown	2.5s	auto-advance to scripted conversation
Chat -> Decision	last scripted chat bubble appears	manual/after typing pause	show bottom sheet
Decision -> Result	button tap	instant	apply branch logic and persist
Result -> Sync	user dismisses result modal	manual	show mock sync overlay

12. Content ownership

All scammer dialogue, the RBI-style educational text, and every label used in the demo should live in one content file so the presentation team can tweak copy without touching component code.

```
export const content = {
  dashboard: {
    balanceLabel: "Balance",
    trustLabel: "Trust Score",
  },
  reward: {
    title: "Congratulations! You have won a UPI Reward of ₹2,000. Claim immediately.",
    subtitle: "Offer expires soon.",
  },
  chat: {
    messages: [
      { id: "m1", role: "npc", text: "Hello! You are eligible for a reward." },
      { id: "m2", role: "npc", text: "Verify quickly to avoid account issues." },
      { id: "m3", role: "npc", text: "Click this link to claim your reward." },
    ],
    warning: "Caution. Real banks never ask you to click links to receive money.",
  },
};
```

13. Demo hardening checklist

- Use no network-dependent assets during the live pitch.
- Preload images and Lottie JSON before starting the demo.
- Disable developer-only debug logs on the presentation build.
- Keep transitions deterministic; avoid randomness entirely.
- Test the complete flow on the weakest Android device available.

- Have a one-tap reset that restores balance to ₹5,000 and Trust Score to 80/100.
- Keep all copy locked unless the team explicitly changes the PRD narrative.

14. Final recommendation

Build the app as a single scripted experience with tiny reusable UI atoms, one context provider, and one storage service. That gives you something stable enough for a live hackathon demo and simple enough that you can still fix it at 2 a.m. without crying into the terminal.